

Real-Time Transit System

FINAL REPORT

INFO 4190 (S10)

2026-04-25

Group 1:

Chloe	Chang	100441028
Sanjeet	Khangura	100379050
Kumardeep	Singh	100369778
James	Sinclair	100443197

TABLE OF CONTENTS

1	Project Overview	1–4
1.1	Purpose	1–4
1.2	Solution	1–4
1.3	Background Information	1–4
1.3.1	GTFS.....	1–4
2	Requirements	2–5
2.1	Functional	2–5
2.1.1	Data Ingestion & Storage	2–5
2.1.2	Data Processing & Analysis	2–5
2.1.3	Machine Learning & Prediction.....	2–5
2.1.4	User Interface / Web Dashboard.....	2–6
2.1.5	System Administration	2–6
2.2	Non-Functional.....	2–6
2.2.1	Performance Requirements	2–6
2.2.2	Availability & Reliability	2–6
2.2.3	Security & Privacy	2–7
2.2.4	Usability	2–7
3	Literature Review	3–8
3.1	Problem Description	3–8
3.2	Existing Solutions	3–8
3.2.1	Google Maps (Google Transit)	3–8
3.2.2	Transit App.....	3–8
3.2.3	Citymapper	3–8
3.3	Solution Comparison	3–9
3.4	Gap Identification	3–9
3.5	Justification	3–9
3.6	Method	3–10
4	Prototype & Testing.....	4–11
4.1	Interface Design	4–11
4.1.1	Route List	4–11

4.1.2	Route Details View.....	4—12
4.2	Testing	4—13
4.2.1	Usability Testing	4—13
4.2.2	System Testing	4—14
4.2.3	Design Testing	4—14
5	Risks	5—15
6	Summary & Future Work	6—15
6.1	Summary	6—15
6.2	Future Work	6—15

1 PROJECT OVERVIEW

1.1 PURPOSE

The Real Time Transit System is designed provide users of public transportation access to deeper insights into reliability and historical data for the routes they use. It aims to provide commuters, transit advocates, and route planners with detailed historical data, analysis, and future predictions. The system analyses current route conditions to identify in real-time issues such as bus bunching, major route delays (e.g. accidents, detours, abnormal traffic), and missing/unexpected buses.

1.2 SOLUTION

We have designed a system that connects to real-time data feeds of publicly accessible transit data to gather information on the current state of a transit system. This data is stored for future analysis. This stored data allows us to identify trends and patterns and to compute models of how any given route behaves under various conditions. We can then use these models to predict when a given bus will arrive, given the current state of the route/system. These real-time states, historical analyses, and future predictions can be provided to a web dashboard.

This dashboard, accessible to users on both mobile and desktop browsers, will display historical reliability data, current route conditions, and predictions of future arrival times for any given route.

1.3 BACKGROUND INFORMATION

This report assumes that readers already have a basic understanding of some technical concepts.

1.3.1 GTFS

General Transit Feed Specification is a data format originally developed by Google to allow transit agencies to publish data to navigation apps like Google Maps. This typically includes routes and timetables. An extension of GTFS called GTFS Realtime allows agencies to supplement these static schedules with real-time updates. This can include updated arrival times, alerts/detours, and, most importantly, the real-time GPS position of each bus.

2 REQUIREMENTS

This section outlines the functional and non-functional requirements for the system. The functional requirements describe what the system must do, including data collection, processing, prediction, and the user interface. The non-functional requirements define how the system should perform in terms of reliability, performance, security, and usability.

2.1 FUNCTIONAL

2.1.1 Data Ingestion & Storage

1. The system must collect the current state of buses and trips from the transit operator's GTFS-Realtime API endpoints.
2. The system must download and record the current schedule of the transit operator's bus network from the GTFS Static API.
3. The system must normalize and store time-series data from the API endpoints into the database.
4. The system must handle API and system failures gracefully by recording the error and retrying when appropriate, without crashing.
5. The system must clean and filter incoming feeds for invalid or non-sensical data, logging the discrepancy instead of recording the data (e.g. impossible travel).

2.1.2 Data Processing & Analysis

1. The system must calculate Schedule Adherence by comparing the real-time position of buses against their expected positions in the GTFS Static schedule.
2. The system must detect Bus Bunching by measuring whether the headway between two buses meets a threshold / percentage of the typical headway.
3. The system must identify Service Gaps (missing buses) where a trip is never present on the real-time feed.
4. The system must aggregate historical data to create Reliability Profiles for each route and route segment. This should include, but not be limited to, schedule adherence, bus bunching, and service gaps, by hour of day.

2.1.3 Machine Learning & Prediction

1. The system must provide a mechanism to train a predictive model using the collected historical time-series data.
2. The system must generate a Forecasted Arrival Time for upcoming buses by feeding current state data into the trained model.
3. The System must verify prediction accuracy by comparing and logging predicted arrival times against actual times once the trip is completed.

2.1.4 User Interface / Web Dashboard

1. The dashboard must allow users to search for and select specific transit routes.
2. The dashboard must display an indicator summarizing the current state of the selected route.
3. The dashboard must display a comparison visualization showing the scheduled time versus the actual/predicted time for upcoming stops.
4. The dashboard must visualize bus bunching alerts, highlighting where on the route buses are too close together.
5. The dashboard must display historical reliability charts (e.g. a line graph showing the difference between scheduled and actual times over a given time period for a given route.)

2.1.5 System Administration

1. The system must provide a web interface for administrators to view system logs (ingestion status, processing errors) and processing status.
2. The system must allow administrators to manually re-trigger processing of historical data and/or machine learning model re-training.
3. The system must allow administrators to fine-tune parameters and thresholds used by the system to calculate metrics such as reliability, status, and bus bunching.

2.2 NON-FUNCTIONAL

2.2.1 Performance Requirements

1. The system will ingest GTFS data updates and persist it to storage within 30 seconds of the timestamp provided by the TransLink feed.
2. For common user interactions (e.g. viewing a route dashboard), the system will return results within 5 seconds for at least 90% of requests.
3. Live dashboard updates will be displayed within 60 seconds.

2.2.2 Availability & Reliability

1. The dashboard will achieve 99.5% availability during defined operating hours over a one-month evaluation period.
2. If real-time data is unavailable for more than 2 consecutive update intervals, the system will fall back to schedule-based data and display a visible status indicator to users. Missing data is recorded as such to prevent the system using them in its calculations.
3. The system will be modular and resilient, with the highest priority assigned to the collection of transit data. A failure in one part of the system should not prevent the system from continuing to record real-time transit data.

2.2.3 Security & Privacy

1. All system API's and data transfers will use encrypted communication protocols.
2. Administrative system functions will require role-based authorization.
3. The system will not store, or process personally identifiable information related to transit users.

2.2.4 Usability

1. At least 80% of test users will be able to interpret route reliability and delay indicators correctly during usability testing.
2. The dashboard will function correctly on desktop and mobile screens with widths of at least 360 pixels.

3 LITERATURE REVIEW

3.1 PROBLEM DESCRIPTION

Transit agencies such as TransLink provide static schedules that estimate when buses should arrive based on typical travel conditions. In practice, however, buses do not always follow these schedules closely. This can lead to delays, missing buses, bus bunching, and situations where buses appear in tracking apps but do not arrive as expected.

Although transit agencies provide real-time data through GTFS-Realtime feeds, most mapping applications use this data only at a basic level. They do not give users a clear way to compare current route performance with historical patterns or to understand how reliable a route usually is. They also often rely on limited real-time information rather than deeper analysis of past data.

Because of this, there is a need for a system that can collect transit data over time, identify route reliability patterns, and use that information to provide more accurate predictions and clearer route insights through an interactive dashboard.

3.2 EXISTING SOLUTIONS

3.2.1 Google Maps (Google Transit)

Google Maps is a widely used navigation platform that includes transit routing and live bus tracking. It integrates with GTFS data and provides useful directions for point-to-point travel. However, it is mainly designed for navigation rather than route analysis. Users who simply want to check bus times or review the general condition of a route may find it less convenient. It also does not provide historical performance data or route reliability profiles.

3.2.2 Transit App

Transit is a mobile-focused app designed for public transit users. It shows nearby routes, live departure times, and real-time tracking based on the user's location. It also uses crowdsourced data to improve arrival estimates. While this can improve accuracy, it may sometimes produce unreliable predictions when data is incomplete or unusual. The app also includes user ratings, but it does not offer detailed historical analysis.

3.2.3 Citymapper

Citymapper is another transit app that focuses on urban travel and multi-modal commuting. It uses live traffic conditions to improve arrival predictions and provides useful information about nearby stops and routes. However, like the other solutions, it does not present historical route performance or reliability data to users.

3.3 SOLUTION COMPARISON

A table outlining the strengths and weaknesses (in the context of the proposed system) of the three existing solutions is provided in Table 1: Solution Comparison Table (Strengths and Weaknesses).

Table 1: Solution Comparison Table (Strengths and Weaknesses)

Solution	Strengths	Weaknesses
Google Maps	• Web application available	• Automobile focus • Limited handling of schedule variance • Schedules are cumbersome and unintuitive to access
Transit App	• Multi-modal focus • Adapts estimates based on current and historical data • Shows user-surveyed ratings and reliability scores • Allows users to view schedules and estimates for nearby routes	• Mobile application only • Occasionally provides wildly incorrect estimates
Citymapper	• Partial web application available • Multi-modal focus • Estimates travel times using traffic data • Lists nearby bus stops and routes	• No historical data presented to users • Lists only scheduled times for future buses (when bus position and traffic is unknown)

3.4 GAP IDENTIFICATION

Although real-time transit data is available, there is still a gap in how it is presented to users. Most current applications provide schedules, routes, and estimated arrival times, but they do not show historical reliability patterns or explain how a route is performing over time. Users are usually given a prediction without any context.

This leaves commuters, researchers, and transit users without a clear way to understand route reliability or compare current conditions with past behaviour. There is also limited use of machine learning for predicting arrival times based on historical trends. As a result, there is a need for a system that combines real-time data, historical analysis, and predictive modelling in one place.

3.5 JUSTIFICATION

The proposed Real-Time Transit System is intended to address this gap by giving users clearer and more transparent transit information. While existing tools are useful for basic navigation, they do not provide historical context or detailed route reliability data.

This system focuses on turning real-time transit feeds into useful long-term insights. By showing route reliability, delays, and bus bunching trends, it gives users a better understanding of how their route is actually performing. This can help commuters make more informed travel decisions and reduce frustration caused by unreliable service.

3.6 METHOD

The proposed system addresses the identified gap through a multi-layered technical architecture designed to transform transient real-time data into a permanent analytical resource. By comparing longitudinal data against static schedules, our system offers a level of predictive reliability and transparency that current market solutions cannot match.

The methodology for solving this problem relies on the following structural pillars:

- **Data Ingestion Layer:** The system will use a scheduled worker to poll TransLink's GTFS-Realtime API every 30 seconds. Unlike current solutions that discard this data after the vehicle passes, our system will persist these "snapshots," ensuring that valuable contextual information is never lost.
- **Historical Profiling:** Data will be stored in a time-series database which allows the system to calculate "Normalcy Deviations." This methodology allows the system to determine exactly how much a bus's current speed or location differs from its historical average for a highly specific temporal window (e.g., a Tuesday at 5:00 PM).
- **Predictive Modeling:** We will implement a Machine Learning (ML) model (for example, an LSTM or XGBoost regressor) trained on the historical dataset to forecast arrival times. Rather than relying solely on the transit agency's immediate data feed or reactionary crowd-sourced approximations, this model will account for variables that existing apps ignore, such as chronic "bus bunching" patterns at specific intersections.
- **Analytical Dashboard:** The front-end will be an interactive web dashboard. Instead of a simple "minutes away" countdown, users will be provided with a collection of tools, including a "Reliability Score" for their route based on the last 30 days of performance.

4 PROTOTYPE & TESTING

4.1 INTERFACE DESIGN

This section outlines the main interfaces of the transit application and how users interact with them. The design focuses on making route information easy to find and understand, while also providing useful real-time and predictive data.

The following pages describe the key interfaces, including the main Route List page and the detailed Route View page. Each interface is designed to support quick navigation, clear information display, and an overall simple user experience.

4.1.1 Route List

Figure 4-1: Main Page is the main page of the web application, and the first screen users see. It provides an overview of all available bus routes and helps users quickly find the ones they need.

At the top of the page, there is a search bar where users can enter a bus number to find a specific route. This makes it easy to go directly to a route without scrolling through the full list.

Below the search bar, all routes are displayed in a list. Users can filter this list by route number or by area, which helps them focus on buses that are relevant to them or nearby.

Each route includes an icon that shows its current status, such as running normally, slightly delayed, or experiencing a larger issue. This allows users to quickly understand how a route is performing at a glance.

The page also displays any routes the user has saved, making them easy to access without needing to search again.

Overall, this interface is designed to be simple and easy to use, with a focus on helping users find route information quickly.

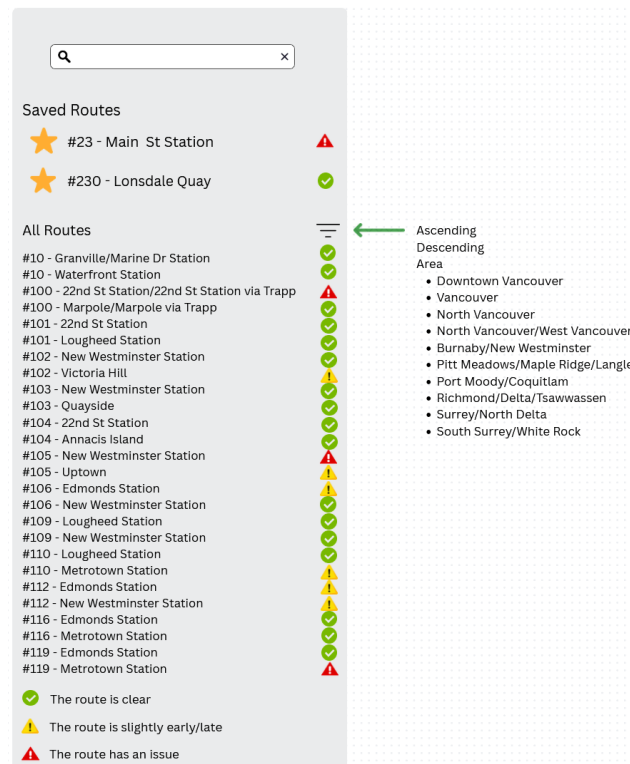


Figure 4-1: Main Page

4.1.2 Route Details View

Figure 4-2: Route Details Page is displayed when a user selects a specific bus route, either by searching or choosing one from the main list. This page provides a detailed overview of the selected route and its current performance.

At the top of the page, the name of the route is shown along with an overall health rating. This rating is based on how well the buses on the route are performing. For example, a 5-star rating indicates that buses are running on time, while a lower rating indicates delays or issues like bus bunching.

If there is a problem on the route, an alert is displayed describing the issue and where it is occurring. For instance, the system may indicate bus bunching between two specific stops, helping users quickly understand what is happening and where.

The page also includes a real-time map that shows the route and the current positions of buses. Buses are coloured (red or green) based on their status, making it easy to spot delays or clustering along the route.

In the second half of the page, users can view both the scheduled arrival times and predicted arrival times for each stop. The predicted times are generated using machine learning based on historical data, providing a more accurate estimate of when the bus will arrive.

Finally, a graph is included to show trends in bus performance over time. It displays how early or late buses tend to be at different times of the day, giving users additional insight into patterns on the route.

Overall, [Figure 4-2: Route Details Page](#) is designed to give users a clear and detailed understanding of a specific route, combining real-time information and predictive data.

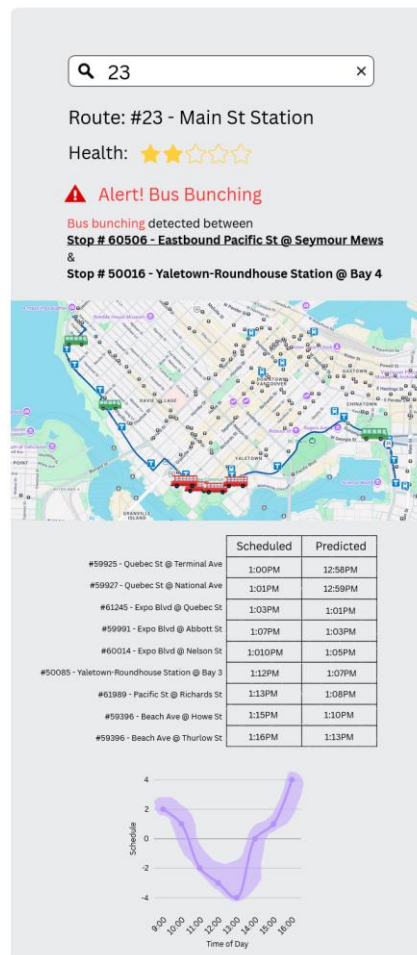


Figure 4-2: Route Details Page

4.2 TESTING

This section outlines the planned testing approach for the application. The purpose of testing is to ensure the system is visually clear, easy to use, and functions correctly with accurate data. Testing will be divided into three areas: design testing, usability testing, and system testing.

4.2.1 Usability Testing

Usability testing will focus on how easy the application is to use from a user perspective. This will involve approximately 3–5 participants completing common tasks such as searching for a route, filtering the route list, and viewing detailed route information.

The main goal is to ensure that users can navigate the system without confusion and can quickly access the information they need. Observations will be made to identify any areas where users experience difficulty or hesitation.

Feedback collected during this stage will be used to improve navigation, simplify interactions, and make key features more intuitive.

4.2.2 System Testing

System and API testing will focus on ensuring that the application correctly retrieves, processes, and displays data. This includes checking that data from TransLink, GTFS, and GTFS-Realtime APIs is properly received and shown in the application.

Testing will also verify that route information, real-time bus locations, and predicted arrival times are accurate and update correctly. In addition, the system will be tested to ensure it can handle cases where API data is missing, delayed, or unavailable.

The accuracy of the prediction system will also be evaluated to ensure that historical data is being used correctly to generate reasonable estimates.

Overall, this stage will ensure that the application functions correctly and provides reliable information to the user.

4.2.3 Design Testing

Design testing will focus on the visual layout and overall structure of the interface. The goal is to ensure the application is clear, consistent, and easy to understand.

This will include checking elements such as spacing, fonts, colours, and icons to make sure they are used consistently across all pages. The layout and structure of each page will also be reviewed to ensure information is organized logically and is easy to scan.

In addition, the overall flow of the interface will be evaluated to ensure that the design supports a simple and intuitive user experience. Feedback from this stage will be used to refine the visual design and improve clarity where needed.

5 RISKS

This section outlines several potential risks that may impact the performance and reliability of the project.

One key risk is the project's dependence on external data sources, including TransLink, GTFS, and GTFS-Real-Time APIs for both scheduled and real-time bus data. Any missing, delayed, or inaccurate data from these sources could affect the accuracy of the route information displayed in the web application.

Another risk is related to prediction accuracy. Since predicted arrival times are based on historical route data, unexpected events such as weather conditions, road closures, or major delays may reduce the accuracy of these predictions.

There is also a risk associated with API and integration issues. If any of the external APIs change, slow down, or become unavailable, it could impact features such as route overviews, timing information, and live bus tracking. In cases where real-time data is unavailable, the system would need to rely only on scheduled and historical data.

Finally, real-time update delays may affect the user experience. Bus location data is not always updated instantly, which means the positions shown on the map may be slightly behind the buses' actual locations.

Overall, these risks highlight the system's reliance on external data and the challenges of providing accurate real-time and predictive information.

6 SUMMARY & FUTURE WORK

6.1 SUMMARY

This project proposed a Real Time Transit System designed to provide users with clearer insight into bus route reliability, real-time conditions, and predicted arrival times. The system combines transit data collection, historical analysis, machine learning predictions, and a web-based dashboard to give users a more detailed view of how routes are performing.

Overall, the project aims to move beyond basic transit tracking by showing not only where buses are, but also how reliable a route is over time. By storing and analyzing historical data, the system can help users identify delays, bus bunching, and other service issues more easily.

6.2 FUTURE WORK

There are several possible improvements that could be added in future versions of the system. One potential extension would be to support more transit agencies and routes beyond the initial target system. This would make the application more useful across a wider area.

Another improvement would be to enhance the prediction model using additional features such as weather, traffic conditions, or special service disruptions. This could improve the accuracy of forecasted arrival times.

Future versions could also include more advanced visualizations, such as route comparisons, pattern summaries, or alerts for recurring service issues. Additional user features, such as saved route notifications or custom alerts, could also improve the experience.

Finally, the system could be expanded with stronger administrative tools and more detailed reporting to help monitor long-term route performance and model accuracy.